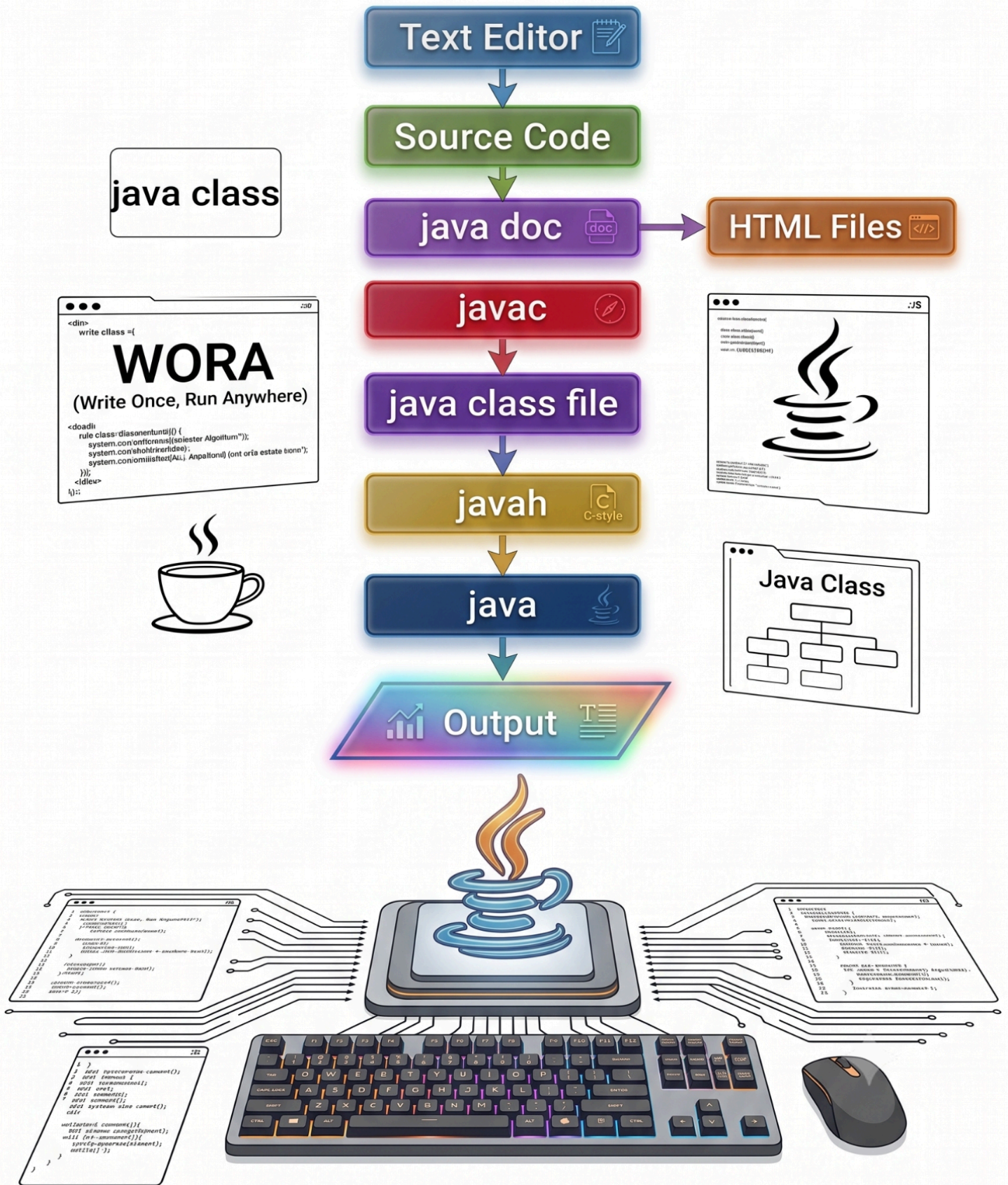


CHAPTER 1

INTRODUCTION TO JAVA



INDEX

SL. No.	Topic	Page No.
1	<u>Java History</u>	3
2	<u>Java Features</u>	3
3	<u>How Java Differs from C and C++</u>	6
4	<u>Difference between Java and C</u>	7
5	<u>Difference between Java and C++</u>	8
6	<u>Java and the Internet</u>	8
7	<u>Java and the WWW</u>	9
8	<u>Web Browser</u>	10
9	<u>Hardware and Software Requirements</u>	10
10	<u>Java Support Systems</u>	11
11	<u>Java Environment</u>	11
12	<u>Application Programming Interface (API)</u>	13
13	<u>Java Runtime Environment (JRE)</u>	13
14	<u>Difference between JRE and JDK</u>	14

INTRODUCTION TO JAVA

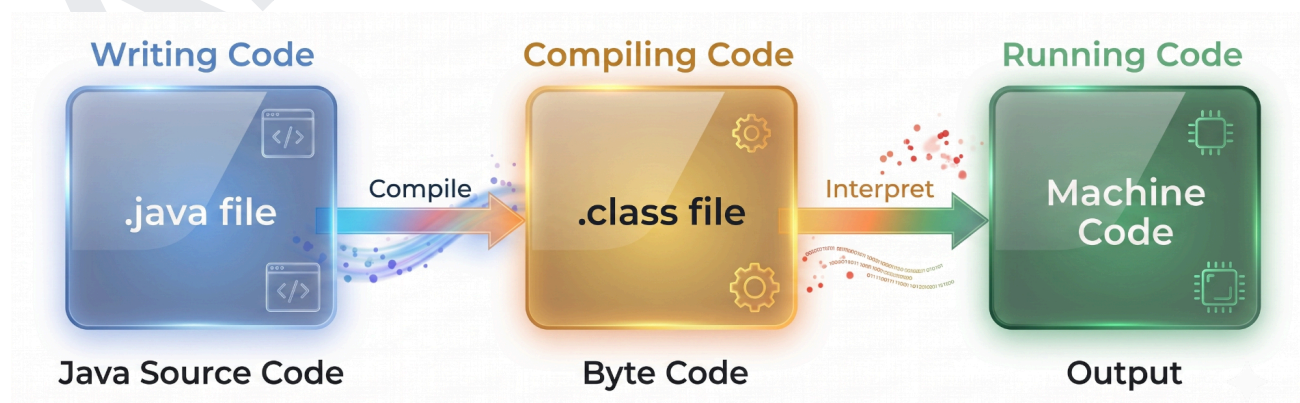
Java History

- **Origin & The Green Team:** The Java language project was initiated in **June 1991** by a team of Sun Microsystems programmers including **James Gosling, Patrick Naughton, Chris Warth, Mike Sheridan, and Ed Frank**. This group was known as the **Green Project team**. Their core objective was to create a platform-independent language capable of building embedded software for consumer electronic devices.
- **Evolution of the Name:**
 - **Oak:** Initially, the language was named "**Oak**" after an oak tree that stood outside James Gosling's office.
 - **Java:** In **1995**, Oak was renamed "**Java**" because "Oak" was already a registered trademark of Oak Technologies. The name was suggested by a team member while sipping Java coffee.
- **Official Release:** Java was released in **1995** by Sun Microsystems as a general-purpose, Object-Oriented Programming (OOP) language. It was designed to run across diverse platforms such as **Windows, Mac OS, and UNIX**.

Key Features of Java

1. Compiled and Interpreted

- **Concept:** Java uses a two-stage system combining compilation and interpretation. First, the Java compiler translates source code into an intermediate format called **Bytecode** (`.class` file). Second, the Java Interpreter—known as the **Java Virtual Machine (JVM)**—translates the Bytecode into machine code executed directly by the computer.
- **Real-World Analogy:** Imagine writing a document in English, translating it into an international intermediate guide, and then having local interpreters convert that guide into local regional languages on demand.

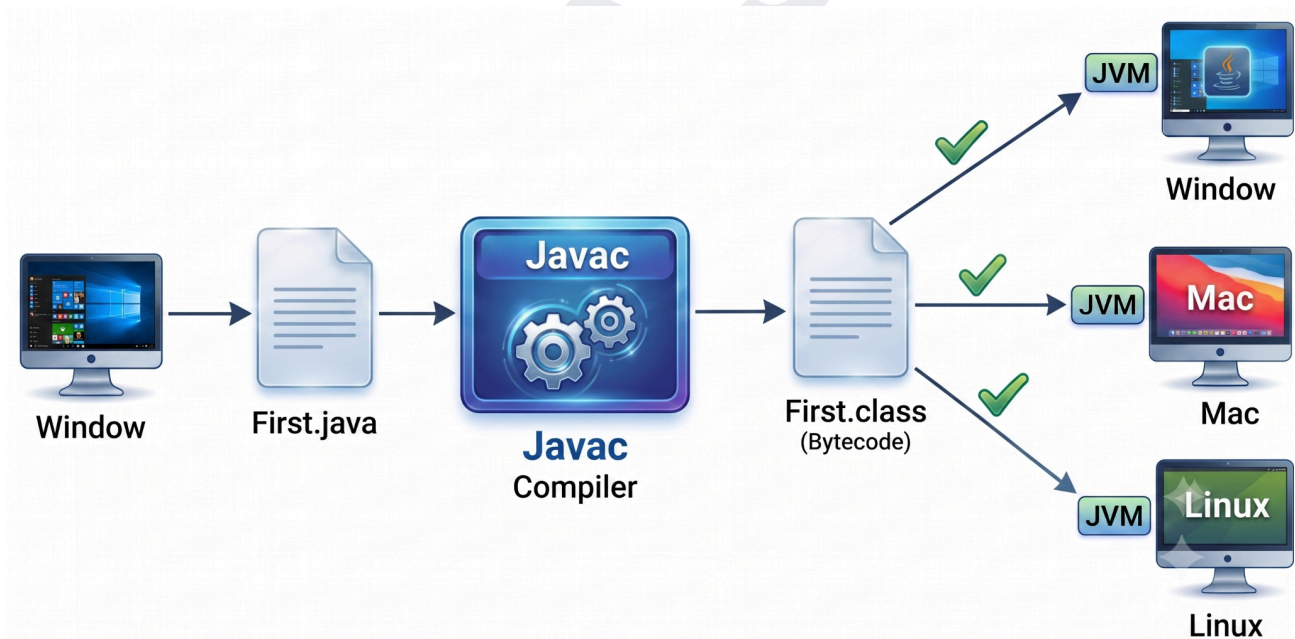


2. Object-Oriented

- **Concept:** Java is a pure object-oriented language where software is organized in a modular structure combining data and behaviors. The basic concepts of OOP in Java are:
 - **Object**
 - **Class**
 - **Inheritance**
 - **Polymorphism**
 - **Abstraction**
 - **Encapsulation**
- **Real-World Analogy:** In manufacturing, a blueprint represents a **Class**, while every physical car produced from that blueprint is an **Object** with specific properties (color, speed) and actions (accelerate, brake).

3. Platform Independent

- **Concept:** Java code can run across multiple platforms (e.g., Windows, Linux, Mac/OS, Sun Solaris) without needing platform-specific modifications. Because code compiles into intermediate Bytecode executed by system-specific JVMs, Java follows the **Write Once, Run Anywhere (WORA)** principle.
- **Real-World Analogy:** A standard PDF file opens and looks identical whether viewed on an iPhone, Android device, Windows PC, or Mac.



4. Portable

- **Concept:** Java programs can easily be transferred from one computer system to another. Portability is guaranteed because Java bytecode works across any machine with a JVM, and the memory size of primitive data types remains machine-independent.
- **Real-World Analogy:** A standard USB drive works identically when plugged into any brand of computer or operating system.

5. Simple

- **Concept:** Java syntax is easy to learn and clean. It intentionally omits complex, confusing, or error-prone features found in earlier languages, such as:
 - Operator overloading
 - Multiple inheritance
 - Explicit memory allocation
 - Pointers
- **Real-World Analogy:** Driving an automatic vehicle compared to a manual transmission car—unnecessary manual controls are eliminated so you can focus on driving.

6. Robust

- **Concept:** Java is designed for high reliability through strict safety mechanisms:
 - Strong memory management
 - Elimination of pointers to avoid security risks
 - **Automatic Garbage Collection** via the JVM to clear unused memory objects
 - Built-in type-checking and exception handling mechanisms
- **Real-World Analogy:** An automated kitchen appliance that constantly cleans up mess as it works and catches spills before they damage the counter.

7. Secure

- **Concept:** Java provides a virtual firewall between the application and the host operating system. Java programs execute inside a confined environment called the **Java Runtime Environment (JRE)**, preventing unauthorized access to underlying system resources.
- **Real-World Analogy:** A web browser running web applications inside isolated browser tabs so malicious sites cannot access your computer's personal hard drive files.

8. Multithreaded

- **Concept:** Java natively supports multithreading, allowing programs to execute multiple tasks simultaneously. This design allows developers to build smooth, interactive applications.
- **Real-World Analogy:** Streaming video online while simultaneously downloading a file and chatting with friends—all happening at the same time without freezing your screen.

9. High-Performance

- **Concept:** Java achieves high speed by employing a **Just-In-Time (JIT) compiler** inside the JVM. The JIT compiler converts Java Bytecode directly into machine language instructions on the fly, making execution much faster than pure interpretation.
- **Real-World Analogy:** A translator who memorizes frequently used phrases during a speech to instantly speak them without pausing to look up dictionary definitions.

10. Distributed

- **Concept:** Java supports building applications distributed across interconnected computer systems via the internet. Using frameworks like **Remote Method Invocation (RMI)** and **Enterprise Java Beans (EJB)**, objects on one JVM can call and execute procedures on a remote JVM.
- **Real-World Analogy:** Global cloud banking or airline reservation systems where servers across different cities coordinate to process transactions in real time.

11. Dynamic

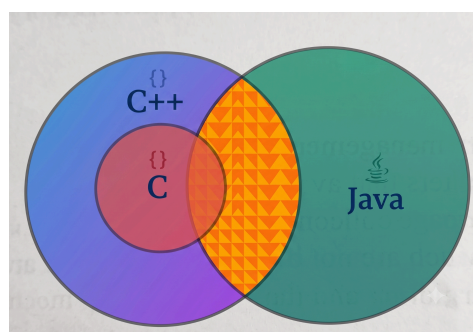
- **Concept:** Java is designed to adapt to evolving environments. It preserves extensive runtime information used to verify and resolve object access while the program is running.
- **Real-World Analogy:** Smartphone apps that dynamically update or download extra modules as needed without requiring you to reinstall the entire application.

12. Architecture-Neutral

- **Concept:** The Java compiler produces an architecture-neutral object file format (Bytecode), ensuring compiled code can execute across various processor architectures as long as a Java runtime system is installed.
- **Real-World Analogy:** Universal audio files (like MP3s) that play seamlessly on ARM chips, x86 chips, or specialized sound hardware.

How Java Differs from C and C++

- **Language Type:** Java is a **true object-oriented language**, whereas C++ is essentially C with object-oriented extensions.
- **Backward Compatibility:** C++ maintains backward compatibility with C, meaning old C programs can run under C++. Java does not maintain backward compatibility with C or C++.
- **Complexity:** Although Java shares similar syntax with C++ regarding object-oriented extensions, it eliminates complex C++ features that make code difficult to follow and maintain.
- **Relationship:** C++ is a **superset of C**, but Java is **neither a superset nor a subset** of C or C++.



Differences Between Java and C

Feature / Aspect	Java	C
Programming Paradigm	Object-Oriented Programming Language.	Procedure-Oriented Programming Language.
Pointers, Structures, & Unions	Does not support pointers, structures, or unions.	Supports pointers, structures, and unions.
Variable Declarations	Local variables can be declared anywhere inside a method or block before use.	Local variables must be declared at the beginning of a method or block.
Execution Method	Interpreted language that compiles source code into bytecode .	Compiled language that compiles source code directly into native machine code .
Design Approach	Uses a bottom-up approach .	Uses a top-down approach .
Type Modifiers	Does not support type modifiers.	Supports type modifiers.
Method Overloading	Supports method overloading.	Does not support method overloading.
Preprocessors	Does not support preprocessors.	Supports preprocessors.
Jump Statements	No goto statement (though goto and const are reserved keywords).	Supports goto statements.

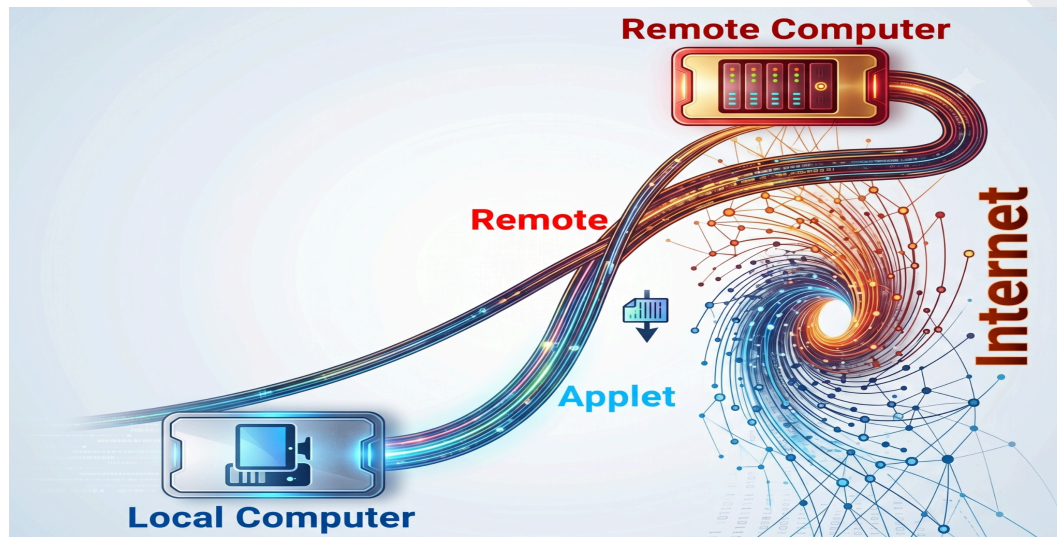
Differences Between Java and C++

Feature / Aspect	Java	C++
Platform Independence	Interpreted bytecode ensures platform independence across systems.	Generates object code that may not run on different platforms.
Language Constructs	Does not support pointers, templates, unions, operator overloading, or structures.	Supports pointers, templates, unions, operator overloading, and structures.
Control & Cleanup	No <code>goto</code> statements and no destructors .	Supports <code>goto</code> statements and destructors (invoked automatically upon object destruction).
Inheritance & Threads	Does not support direct multiple inheritance ; includes built-in support for threads .	Supports direct multiple inheritance ; has no built-in support for threads .
Compilation & Scope	No conditional compilation , conditional inclusion, or scope resolution operator (<code>::</code>).	Supports conditional compilation , conditional inclusion, and the scope resolution operator (<code>::</code>).
Documentation & Exceptions	Provides built-in support for documentation comments (<code>/** ... */</code>) and enforces <code>try/catch</code> for declared thrown exceptions.	Does not support documentation comments and does not require <code>try/catch</code> even if an exception is thrown.

Java and the Internet

- **Connection to the Web:** Java is **strongly associated with the Internet**. It allows developers to write **applet programs** that users can run locally through a **Java-enabled browser** (such as HotJava).
- **Remote Execution:** Users can download an applet hosted on any remote server across the Internet and execute it on their local machine, turning the Internet into an extension of local computer storage.
- **Web Integration (`<APPLET>` Tag):**

1. A user requests an HTML document containing an **<APPLET>** tag from a remote web server.
2. The server sends the HTML document, and the corresponding **applet bytecode** is transferred to the user's computer.
3. The local **Java-enabled browser** interprets the bytecode to render the output.
4. Once downloaded, the user can continue interacting with the applet locally without additional downloads from the remote server.



Java and the World Wide Web

- **Overview of the Web:** The World Wide Web (WWW) is an open-ended information retrieval system operating within the Internet's distributed environment. Web pages use HTML tags to enable finding, retrieving, manipulating, and displaying documents globally.
- **The **<APPLET>** Tag:** Java communicates with a Web page through a specialized HTML tag called **<APPLET>**.
- **Step-by-Step Web Communication Process:**
 1. **Document Request:** The user's computer sends a request for an HTML document to a remote server.
 2. **HTML Retrieval:** The server accepts the request and returns the HTML document containing the **<APPLET>** tag to the user's browser.
 3. **Bytecode Transfer:** The compiled applet **Bytecode** (generated earlier by the Java compiler) is transferred from the server to the user's computer.
 4. **Local Execution:** A Java-enabled web browser on the user's computer interprets the Bytecode and renders the interactive output.
 5. **Direct Interaction:** The user can continue interacting with the applet locally without triggering additional server downloads.

Web Browsers

- **Definition:** A Web browser is a software application used to locate, retrieve, and display content—including web pages, images, and videos—on the World Wide Web.
- **Client/Server Model:** The browser operates as a **client** on the user's computer, requesting data from a remote **Web server** and displaying the results locally.
- **Key Browsers Highlighted:**
 - **HotJava:** A modular, extensible web browser created by Sun Microsystems and written entirely in Java. Released on **June 1, 1995**, it was the first browser capable of supporting Java applets.
 - **Google Chrome:** An open-source web browser released by Google on **September 2, 2008** (beta version for Windows in 43 languages) to run web applications.
 - **Mozilla Firefox:** A free and open-source browser developed by the Mozilla Foundation (originally created in **2002** under the project name "Phoenix").
- **Core Browser Features:**
 - **Tabbed Browsing:** Opens multiple web resources simultaneously across windows or tabs.
 - **Security & Cleanup:** Supports HTTP Secure (HTTPS) and allows easy clearing of cache, cookies, download history, and search history.
 - **Content Management:** Handles text, images, and video, while offering options to save web pages locally or email page contents.

Hardware and Software Requirements

- **Supported Platforms:** Java is supported across Windows (XP and higher), Sun Solaris, Macintosh, and UNIX operating systems.
- **Minimum System Requirements Table:**

Category	Specification Requirement
Processor	Any 32-bit Architecture processor or higher
RAM (Memory)	256 MB or more
Hard Disk	10 GB minimum space
Operating System	Windows, MAC / Macintosh, or Linux
Programming Tool	JAVA (JDK - Java Development Kit)
User Interface	HTML, APPLET
Web Browsers	Internet Explorer (IE), Mozilla Firefox, Google Chrome

Java Support Systems

Executing Java applications and applets across the Internet relies on several interconnected hardware, software, and network systems:

- **Internet Connection:** Connects the local workstation to the global network.
- **Web Server:** A remote software application that receives client requests for web documents and delivers the required files.
- **Web Browser:** A client application that accesses the Web and executes embedded Java applets locally.
- **HTML:** The standard markup language used to structure content on the Web.
- **<APPLET> Tag:** A specialized HTML tag embedded in web pages to reference and execute Java applets.
- **Java Code:** The source code written by developers to define applet logic.
- **Bytecode:** The compiled, intermediate Java code referenced inside the **<APPLET>** tag and downloaded to the user's computer.
- **Proxy Server:** An intermediate server situated between the client workstation and the target web server to enhance network security.
- **Mail Server:** A dedicated computer system that facilitates routing, sending, and receiving electronic mail.

The Java Environment & JDK

The **Java Environment** comprises two foundational parts: software development tools provided in the **Java Development Kit (JDK)** and pre-built classes/methods organized inside the **Java Standard Library (JSL)**, commonly called the **Application Programming Interface (API)**.

Java Development Kit (JDK) Tools

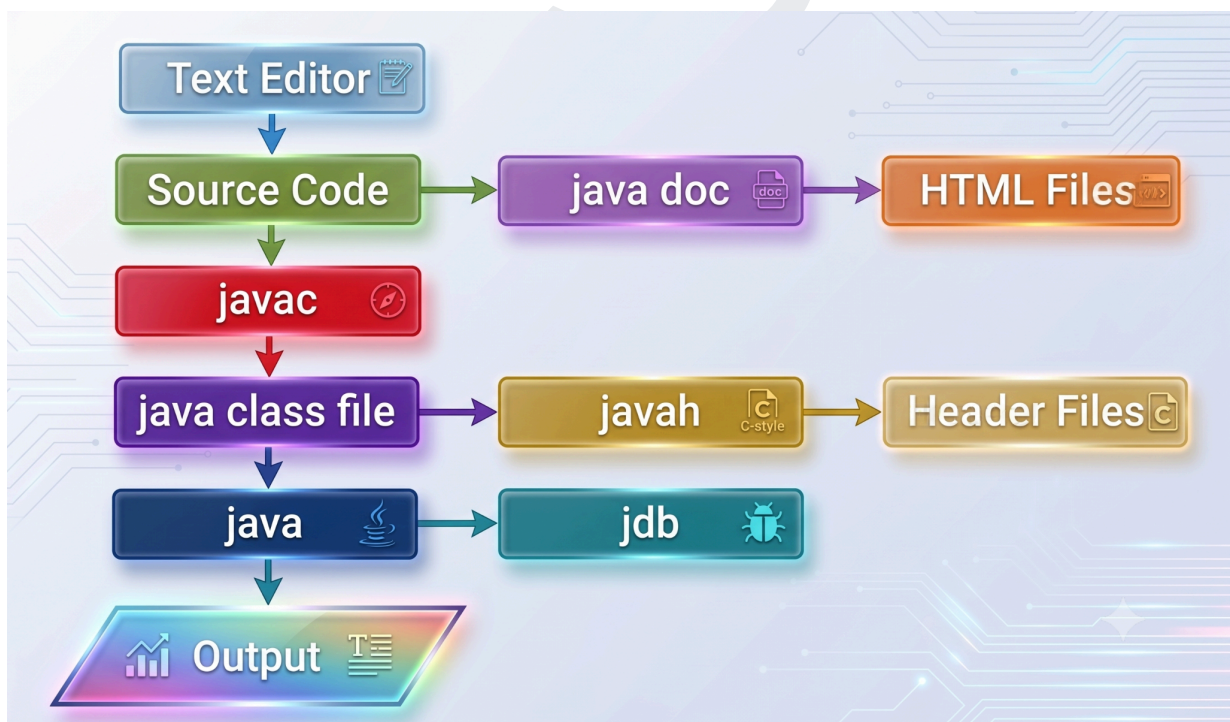
The JDK supplies a suite of command-line utilities used throughout the application lifecycle:

Tool	Description & Primary Function
javac	The Java Compiler that converts human-readable Java source code into bytecode files.
java	The Java Interpreter that reads and executes bytecode files for applications and applets.
javadoc	Automatically generates HTML-formatted documentation from source code comments.

javap	The Java Disassembler that converts compiled bytecode back into a program description.
appletviewer	Runs and tests Java applets directly without opening a full web browser.
jdb	The Java Debugger used to detect, trace, and fix errors in programs.
javah	Produces C-style header files for connecting Java with native code written in C or C++.

Program Build & Execution Workflow

1. **Creation:** Source code is written in a text editor as a **.java** file.
2. **Compilation:** The **javac** compiler translates the **.java** file into a **.class** bytecode file.
3. **Documentation & Headers:** The **javadoc** tool extracts documentation into HTML files, while **javah** creates C header files from class files.
4. **Execution & Debugging:** The **java** interpreter executes the bytecode to produce program output, while **jdb** is used if runtime errors occur.



Application Programming Interface - API

The **Java API** (Java Standard Library) provides hundreds of pre-written classes and methods organized into core **packages**:

- **java.lang (Language Support Package)**: The default automatically imported package containing core language classes and methods.
- **java.util (Utilities Package)**: Contains utility classes for handling tasks like dates, time, and data structures.
- **java.io (Input/Output Package)**: Includes classes for managing system input, output, and file operations.
- **java.net (Networking Package)**: Provides classes for communicating with remote systems over the Internet.
- **java.awt (Abstract Window Toolkit)**: Supplies GUI components for building platform-independent user interfaces.
- **java.applet (Applet Package)**: Provides specialized classes needed to construct web-based Java applets.

Java Runtime Environment - JRE

The **Java Runtime Environment (JRE)** is the execution engine portion of the JDK containing the minimum tools and libraries needed to run compiled Java applications.

Core JRE Components

- **Java Class Libraries**: Dynamically loadable runtime libraries called by Java applications.
- **Java Class Loader**: Loads class files and links them with core class libraries during program execution.
- **Java Virtual Machine (JVM)**: Manages program execution and handles memory and hardware resources.
- **Deployment Technologies**: Includes Java Web Start and the Java Plug-in.
- **UI Toolkits**: Includes graphical rendering toolkits like Java 2D.
- **Integration & Base Libraries**: Includes JDBC (Database Connectivity), JNDI (Naming and Directory Interface), JAR (Java Archive), JNI (Native Interface), JMX (Management Extensions), and JAX-WS (XML Processing).

Difference Between JRE and JDK

Feature / Aspect	Java Runtime Environment (JRE)	Java Development Kit (JDK)
Definition	Java Runtime Environment.	Java Development Kit.
Primary Role	An implementation of the JVM used to run Java applications.	A full software bundle used to develop and run Java applications.
Target Audience	Intended for end users who only need to execute Java programs.	Required by software developers writing Java programs.
Disk Overhead	Smaller software footprint requiring less disk space .	Larger software footprint requiring more disk space .
Included Contents	Includes the JVM, core class libraries, and runtime components .	Includes the entire JRE plus development tools (<code>javac</code> , <code>appletviewer</code> , APIs, debugging tools).

THANK YOU